



# RATINGS ΤΑΙΝΙΩΝ

Αλεξάνδρα Σταθοπούλου 8190156

Σταύρος Σιώπης 8190320

Διδάσκων: Δ. Χατζηαντωνίου

# Περιεχόμενα

|                                                     |    |
|-----------------------------------------------------|----|
| Περιγραφή dataset .....                             | 2  |
| Καθαρισμός dataset.....                             | 5  |
| Μετατροπή αρχείων σε CSV .....                      | 5  |
| <b>Διαχωρισμός έτους Κυκλοφορίας</b> .....          | 7  |
| <b>Έλεγχος Κενών Τιμών</b> .....                    | 7  |
| <b>Έλεγχος Διπλοτύπων Τιμών</b> .....               | 7  |
| <b>Αφαίρεση Τίτλων σε Παρενθέσεις</b> .....         | 8  |
| Δημιουργία Απαραίτητων Πινάκων .....                | 8  |
| <b>Δημιουργία Αρχείου Πλήρους Ημερομηνίας</b> ..... | 8  |
| <b>Δημιουργία Αρχείου Είδους Ταινίας</b> .....      | 9  |
| <b>Δημιουργία Αρχείου Επαγγέλματος Χρήστη</b> ..... | 9  |
| <b>Δημιουργία Αρχείου Ηλικίας Χρήστη</b> .....      | 9  |
| <b>Δημιουργία Αρχείου Φύλλου Χρήστη</b> .....       | 9  |
| <b>Δημιουργία Αρχείου Διασυνδέσεων</b> .....        | 10 |
| Εισαγωγή dataset σε Data Warehouse .....            | 11 |
| Δημιουργία Cube .....                               | 13 |
| Οπτικοποίηση με Tableau .....                       | 16 |

## Περιγραφή dataset

Επιλέξαμε το συγκεκριμένο dataset λόγω της κοινής μας αγάπης για τον κινηματογράφο και του ενδιαφέροντος μας στις αξιολογήσεις και την κριτική ταινιών. Χρησιμοποιήσαμε το **MovieLens 1M Dataset** από το site της [GroupLens](https://grouplens.org) του Πανεπιστημίου της Μινεσότα.

Το dataset περιέχει πάνω από **1 εκατομμύριο αξιολογήσεις** που καταγράφηκαν από **6.000 χρήστες** για περίπου **4.000 ταινίες**. Οι αξιολογήσεις καλύπτουν την περίοδο από το **2000** έως το **2003**, παρέχοντας πλούσια δεδομένα για ανάλυση. Μπορείτε να βρείτε το dataset στον ακόλουθο σύνδεσμο: [MovieLens 1M Dataset](https://grouplens.org/datasets/movielens/1m/).

## MovieLens 1M Dataset

MovieLens 1M movie ratings. Stable benchmark dataset. 1 million ratings from 6000 users on 4000 movies. Released 2/2003.

- [README.txt](#)
- [ml-1m.zip](#) (size: 6 MB, [checksum](#))

Permalink: <https://grouplens.org/datasets/movielens/1m/>

Το συγκεκριμένο dataset περιλαμβάνει δεδομένα που είναι κατανομημένα σε τρία διαφορετικά αρχεία και παρέχουν πληροφορίες σχετικά με τις αξιολογήσεις ταινιών, τους χρήστες και τις ταινίες. Τα αρχεία είναι τα εξής:

1. **ratings.dat** (1,000,209 εγγραφές): Περιέχει τις κριτικές χρηστών για ταινίες και ακολουθεί το παρακάτω format:

**UserID::MovieID::Rating::Timestamp**

2. **users.dat** (6,040 εγγραφές): Περιέχει τα δημογραφικά χαρακτηριστικά των χρηστών και ακολουθεί το παρακάτω format:

**UserID::Gender::Age::Occupation::Zip-code**

3. **movies.dat** (3,883 εγγραφές): Περιέχει λεπτομέρειες για τις ταινίες από το IMDb και ακολουθεί το παρακάτω format:

**MovieID::Title::Genres**

**Πιο συγκεκριμένα, τα δεδομένα περιλαμβάνουν:**

1. **Title:** Ο τίτλος της ταινίας όπως εμφανίζεται στο IMDb, περιλαμβάνοντας εντός παρενθέσεων το έτος κυκλοφορίας της.
2. **Genres:** Τα είδη της ταινίας, διαχωρισμένα με τον χαρακτήρα «|».

Οι πιθανές τιμές των ειδών είναι οι εξής:

- Action
- Adventure
- Animation
- Children's
- Comedy
- Crime
- Documentary
- Drama
- Fantasy
- Film-Noir
- Horror
- Musical
- Mystery
- Romance
- Sci-Fi
- Thriller
- War
- Western

3. **Rating:** Η αξιολόγηση της ταινίας από κάθε χρήστη.

Οι τιμές κυμαίνονται από 1 έως 5 (ακέραιοι αριθμοί).

4. **MovieID:** Αναγνωριστικό της ταινίας.

Οι τιμές κυμαίνονται από 1 έως 3952 (ακέραιοι αριθμοί).

5. **UserID:** Αναγνωριστικό του χρήστη.

Οι τιμές κυμαίνονται από 1 έως 6040 (ακέραιοι αριθμοί).

6. **Gender:** Το φύλο του χρήστη.

Οι πιθανές τιμές είναι:

- M: Άνδρας (Male)
- F: Γυναίκα (Female)

7. **Age:** Η ηλικιακή κατηγορία του χρήστη.

Οι πιθανές τιμές είναι οι εξής:

- 1: "Under 18"
- 18: "18-24"
- 25: "25-34"
- 35: "35-44"
- 45: "45-49"
- 50: "50-55"
- 56: "56+"

8. **Occupation:** Το επάγγελμα του χρήστη.

Οι πιθανές τιμές είναι οι εξής:

- 0: "other" ή not specified
- 1: "academic/educator"
- 2: "artist"
- 3: "clerical/admin"
- 4: "college/grad student"
- 5: "customer service"
- 6: "doctor/health care"
- 7: "executive/managerial"
- 8: "farmer"
- 9: "homemaker"
- 10: "K-12 student"

- 11: "lawyer"
- 12: "programmer"
- 13: "retired"
- 14: "sales/marketing"
- 15: "scientist"
- 16: "self-employed"
- 17: "technician/engineer"
- 18: "tradesman/craftsman"
- 19: "unemployed"
- 20: "writer"

9. **Timestamp:** Η χρονική σήμανση της στιγμής που έγινε η αξιολόγηση.

10. **Zip-Code:** Ο ταχυδρομικός κώδικας του χρήστη.

## Καθαρισμός dataset

Προτού εισάγουμε τα δεδομένα μας στο Data Warehouse, πρέπει να εξετάσουμε και να ελέγξουμε τις τιμές των βασικών χαρακτηριστικών (features), να αφαιρέσουμε εκείνα που δεν θα μας εξυπηρετήσουν ή που περιέχουν λάθη, καθώς και τις ακραίες (outliers) και κενές (null) τιμές. Αρχικά, πλοηγηθήκαμε και εξετάσαμε τα αρχεία μας και τα δεδομένα τους (στήλες και γραμμές του dataset) για να εξοικειωθούμε με αυτά.

Για την ανάλυση και διαχείριση των δεδομένων χρησιμοποιήσαμε το Jupyter Notebook, ένα open-source εργαλείο που χρησιμοποιείται συχνά για ανάλογους σκοπούς.

## Μετατροπή αρχείων σε CSV

Τα αρχεία του dataset ήταν σε μορφή .dat. Για να τα μετατρέψουμε σε .csv, δημιουργήσαμε μια μέθοδο σε Python στο Jupyter Notebook.

```
[4]: convert_dat_to_csv(movie_dat, movie_file)
      convert_dat_to_csv(rating_dat, rating_file)
      convert_dat_to_csv(user_dat, user_file)

File converted successfully: C:\Users\30698\OneDrive\Desktop\ml-1m\movies.csv
File converted successfully: C:\Users\30698\OneDrive\Desktop\ml-1m\ratings.csv
File converted successfully: C:\Users\30698\OneDrive\Desktop\ml-1m\users.csv
```

```
[3]: import csv

def convert_dat_to_csv(dat_file_path, csv_file_path):
    try:
        with open(dat_file_path, 'r') as dat_file:
            lines = dat_file.readlines() # Read all lines from the .dat file

        with open(csv_file_path, 'w', newline='') as csv_file:
            writer = csv.writer(csv_file)
            for line in lines:
                # Split the line using ':' as the delimiter
                writer.writerow(line.strip().split(':'))

        print(f"File converted successfully: {csv_file_path}")
    except FileNotFoundError:
        print("The specified file was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")
```

Έπειτα, δημιουργήσαμε το αντίστοιχο DataFrame για την καλύτερη οπτικοποίηση και ξεκινήσαμε τη διαδικασία του καθαρισμού και της επεξεργασίας.

```
[5]: import pandas as pd

# Read the file with column names
movie_df = pd.read_csv(movie_file, encoding='ISO-8859-1', names=['Movie ID', 'Title', 'Genre'], header=None)

# Reset index to start from 1
movie_df.index = range(1, len(movie_df) + 1)

movie_df
```

```
[5]:
```

|   | Movie ID | Title                              | Genre                        |
|---|----------|------------------------------------|------------------------------|
| 1 | 1        | Toy Story (1995)                   | Animation Children's Comedy  |
| 2 | 2        | Jumanji (1995)                     | Adventure Children's Fantasy |
| 3 | 3        | Grumpier Old Men (1995)            | Comedy Romance               |
| 4 | 4        | Waiting to Exhale (1995)           | Comedy Drama                 |
| 5 | 5        | Father of the Bride Part II (1995) | Comedy                       |

Ακολουθώντας την ίδια διαδικασία, εργαστήκαμε και με τα υπόλοιπα αρχεία, δημιουργώντας DataFrames για καλύτερη ανάλυση και οπτικοποίηση.

| User ID | Movie ID | Rating | Timestamp | User ID | Gender | Age | Occupation | Zip-Code |
|---------|----------|--------|-----------|---------|--------|-----|------------|----------|
| 1       | 1193     | 5      | 978300760 | 1       | 2      | 1   | 10         | 48067    |
| 1       | 661      | 3      | 978302109 | 2       | 1      | 56  | 16         | 70072    |
| 1       | 914      | 3      | 978301968 | 3       | 1      | 25  | 15         | 55117    |
| 1       | 3408     | 4      | 978300275 | 4       | 1      | 45  | 7          | 02460    |
| 1       | 2355     | 5      | 978824291 | 5       | 1      | 25  | 20         | 55455    |
| ...     | ...      | ...    | ...       | ...     | ...    | ... | ...        | ...      |

## Διαχωρισμός έτους Κυκλοφορίας

Χρειαζόμασταν το έτος κυκλοφορίας ως ξεχωριστό attribute, οπότε το διαχωρίσαμε σε ξεχωριστή στήλη στο DataFrame χρησιμοποιώντας κανονικές εκφράσεις (regex).

```
[7]: # Extract the year from the title using a regular expression
movie_df['Year'] = movie_df['Title'].str.extract(r'\((\d{4})\)', expand=False)

# Remove the year from the Title column
movie_df['Title'] = movie_df['Title'].str.replace(r'\s*(\d{4})', '', regex=True)

movie_df
```

```
[7]:
```

|     | Movie ID | Title                       | Genre                        | Year |
|-----|----------|-----------------------------|------------------------------|------|
| 1   | 1        | Toy Story                   | Animation Children's Comedy  | 1995 |
| 2   | 2        | Jumanji                     | Adventure Children's Fantasy | 1995 |
| 3   | 3        | Grumpier Old Men            | Comedy Romance               | 1995 |
| 4   | 4        | Waiting to Exhale           | Comedy Drama                 | 1995 |
| 5   | 5        | Father of the Bride Part II | Comedy                       | 1995 |
| ... | ...      | ...                         | ...                          | ...  |

## Έλεγχος Κενών Τιμών

Στη συνέχεια, ελέγξαμε όλα τα DataFrames για κενές τιμές (NaN) και παρατηρήσαμε ότι δεν υπήρχαν κενές τιμές σε κανένα από τα αρχεία.

```
[9]: movie_df.isna().any() [12]: rating_df.isna().any() [17]: user_df.isna().any()
```

| [9]: | Movie ID    | False | [12]: | User ID     | False | [17]: | User ID    | False |
|------|-------------|-------|-------|-------------|-------|-------|------------|-------|
|      | Title       | False |       | Movie ID    | False |       | Gender     | False |
|      | Genre       | False |       | Rating      | False |       | Age        | False |
|      | Year        | False |       | Timestamp   | False |       | Occupation | False |
|      | dtype: bool |       |       | dtype: bool |       |       | Zip-Code   | False |

## Έλεγχος Διπλοτύπων Τιμών

Στη συνέχεια, ελέγξαμε τα DataFrames για διπλότυπες τιμές και παρατηρήσαμε ότι δεν υπήρχαν.

```
[33]: df.duplicated(subset=['Title', 'Year']).any() [35]: df.duplicated(subset=['User ID', 'Movie ID']).any()
[33]: False [35]: False
```



## Αφαίρεση Τίτλων σε Παρενθέσεις

Παρατηρήσαμε ότι σε ορισμένες ταινίες (270) οι πρωτότυποι τίτλοι εμφανίζονταν σε παρενθέσεις. Δεδομένου ότι αυτές οι πληροφορίες δεν θα μας ήταν χρήσιμες στη συνέχεια της ανάλυσης, αποφασίσαμε να τις αφαιρέσουμε.

```
[10]: # Extract everything within brackets
bracketed_items = movie_df['Title'].str.extractall(r'\((.*?)\)')

# Count the total number of bracketed items
total_count = len(bracketed_items)

# Display the total count
print(f"Total number of bracketed items: {total_count}")

movie_df['Title'] = movie_df['Title'].str.replace(r'\((.*?)\)', '', regex=True)

Total number of bracketed items: 270
```

## Δημιουργία Απαραίτητων Πινάκων

Επιλέξαμε να δημιουργήσουμε όλα τα απαραίτητα (csv) αρχεία στο Jupyter Notebook μέσω Python, ώστε να τα μετατρέψουμε αργότερα στους κατάλληλους πίνακες της βάσης. Συγκεκριμένα, δημιουργήσαμε τα αρχεία **rating\_date**, **movie\_genre**, **user\_age**, **user\_gender**, **user\_occupation** και **associations** για την διευκόλυνση της ανάλυσής μας.

## Δημιουργία Αρχείου Πλήρους Ημερομηνίας

Στα δεδομένα των ratings υπήρχε ένα πεδίο timestamp που υποδείκνυε πότε πραγματοποιήθηκε κάθε αξιολόγηση. Ωστόσο, για την ανάλυσή μας, θέλαμε την ημερομηνία σε κατανοητή μορφή, καθώς και ξεχωριστά πεδία που να περιλαμβάνουν τη χρονιά, τον μήνα, την ημέρα και την ώρα. Γι' αυτό δημιουργήσαμε τον πίνακα **rating\_date**.

```
[26]: data = rating_df['Timestamp']
date_df = pd.DataFrame(data)

# Convert 'Timestamp' column to a full date
date_df['Full Date'] = pd.to_datetime(date_df['Timestamp'], unit='s')

date_df['Year'] = date_df['Full Date'].dt.year
date_df['Month'] = date_df['Full Date'].dt.month
date_df['Day'] = date_df['Full Date'].dt.day
date_df['Hour'] = date_df['Full Date'].dt.hour
date_df['Minute'] = date_df['Full Date'].dt.minute
date_df['Second'] = date_df['Full Date'].dt.second

date_df = date_df.drop(columns=['Timestamp'])
date_df
```

[26]:

|     | Full Date           | Year | Month | Day | Hour | Minute | Second |
|-----|---------------------|------|-------|-----|------|--------|--------|
| 1   | 2000-12-31 22:12:40 | 2000 | 12    | 31  | 22   | 12     | 40     |
| 2   | 2000-12-31 22:35:09 | 2000 | 12    | 31  | 22   | 35     | 9      |
| 3   | 2000-12-31 22:32:48 | 2000 | 12    | 31  | 22   | 32     | 48     |
| 4   | 2000-12-31 22:04:35 | 2000 | 12    | 31  | 22   | 4      | 35     |
| 5   | 2001-01-06 23:38:11 | 2001 | 1     | 6   | 23   | 38     | 11     |
| ... | ...                 | ...  | ...   | ... | ...  | ...    | ...    |

## Δημιουργία Αρχείου Είδους Ταινίας

Σύμφωνα με το README που συνοδεύει το dataset, δημιουργήσαμε το αρχείο **movie\_genre**, ο οποίος περιλαμβάνει όλα τα είδη ταινιών, καθώς και τον κωδικό αριθμό που αντιστοιχεί σε κάθε είδος.

## Δημιουργία Αρχείου Επαγγέλματος Χρήστη

Σύμφωνα με το README που συνοδεύει το dataset, δημιουργήσαμε το αρχείο **user\_occupation**, ο οποίος περιλαμβάνει όλες τις κατηγορίες επαγγελμάτων, καθώς και τον κωδικό αριθμό που αντιστοιχεί σε κάθε επάγγελμα.

## Δημιουργία Αρχείου Ηλικίας Χρήστη

Σύμφωνα με το README που συνοδεύει το dataset, δημιουργήσαμε το αρχείο **user\_age**, ο οποίος περιλαμβάνει όλα τα εύρη ηλικιών, καθώς και τον κωδικό αριθμό που αντιστοιχεί σε κάθε εύρος.

## Δημιουργία Αρχείου Φύλλου Χρήστη

Σύμφωνα με το README που συνοδεύει το dataset, δημιουργήσαμε το αρχείο **user\_gender**, ο οποίος περιλαμβάνει τα φύλλα, καθώς και τον κωδικό αριθμό που αντιστοιχεί σε κάθε φύλλο. Για τη διευκόλυνση μας μετατρέψαμε στο αρχείο users.csv για τα φύλλα όπου M = 1 και όπου F = 2 για να ταιριάζει με το Gender ID του πίνακα user\_gender.

```
# Read the file with column names
user_df = pd.read_csv(user_file, encoding='ISO-8859-1', names=['User ID', 'Gender', 'Age', 'Occupation', 'Zip-Code'],

# Change 'M' to 1 and 'F' to 2, and ensure the values are integers
user_df['Gender'] = user_df['Gender'].map({'M': 1, 'F': 2}).astype(int)

# Reset index to start from 1
user_df.index = range(1, len(user_df) + 1)

user_df
```

```
[224]:
```

|   | User ID | Gender | Age | Occupation | Zip-Code |
|---|---------|--------|-----|------------|----------|
| 1 | 1       | 2      | 1   | 10         | 48067    |
| 2 | 2       | 1      | 56  | 16         | 70072    |
| 3 | 3       | 1      | 25  | 15         | 55117    |
| 4 | 4       | 1      | 45  | 7          | 02460    |
| 5 | 5       | 1      | 25  | 20         | 55455    |

## Δημιουργία Αρχείου Διασυνδέσεων

Για την καλύτερη ανάλυση των δεδομένων μας, αποφασίσαμε να διαχωρίσουμε τα διάφορα είδη των ταινιών, καθώς σε μια ταινία μπορούν να αντιστοιχούν πολλαπλά είδη. Στη συνέχεια, τα εισάγουμε σε έναν πίνακα με τη μορφή:

- MovieID = GenreID (1)
- MovieID = GenreID (2)
- ...

Για να το επιτύχουμε αυτό, χωρίσαμε τα είδη της κάθε ταινίας, τα οποία διαχωρίζονταν από τον χαρακτήρα «|», και τα εισάγαμε στον πίνακα χρησιμοποιώντας ένα νέο AssociationID για κάθε συνδυασμό ταινίας και είδους.

```
[50]: movie_df['Genre'] = movie_df['Genre'].str.split('|')
df_exploded = movie_df.explode('Genre')

associate_df = df_exploded.merge(genre_df, on='Genre', how='left')
associate_df['Association ID'] = associate_df.index + 1

associate_df_f = associate_df[['Association ID', 'Movie ID', 'Genre ID']]

associate_df_f.to_csv(r"C:\Users\30698\OneDrive\Desktop\ml-1m\associations.csv", header=None, index=False)

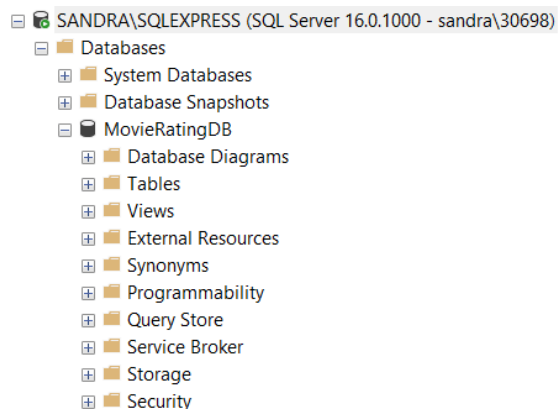
associate_df_f
```

```
[50]:
```

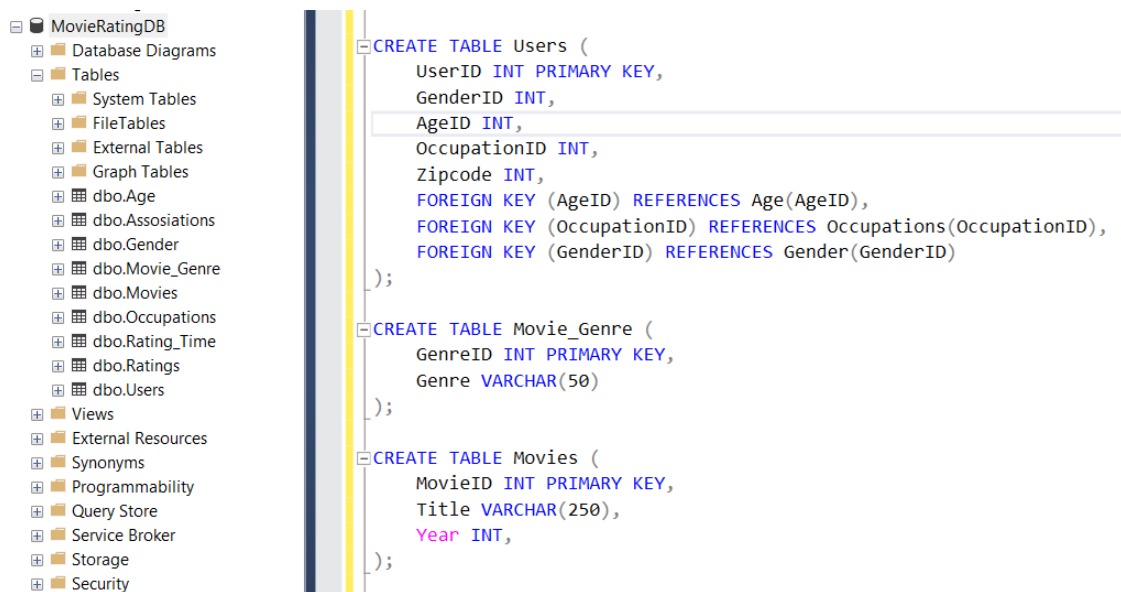
|   | Association ID | Movie ID | Genre ID |
|---|----------------|----------|----------|
| 0 | 1              | 1        | 3        |
| 1 | 2              | 1        | 4        |
| 2 | 3              | 1        | 5        |
| 3 | 4              | 2        | 2        |
| 4 | 5              | 2        | 4        |

## Εισαγωγή dataset σε Data Warehouse

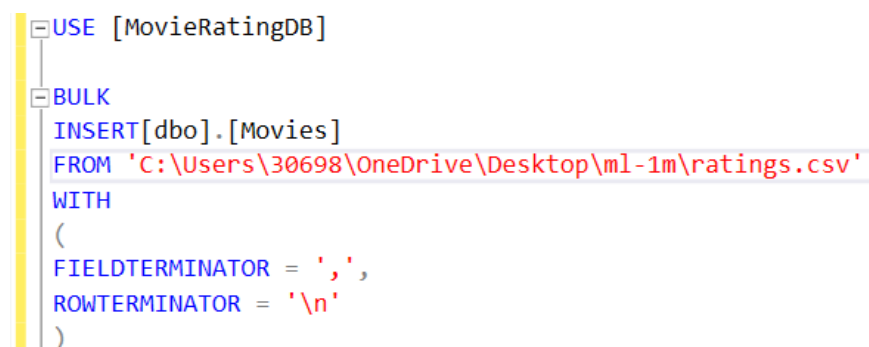
Αρχικά δημιουργήσαμε τη βάση δεδομένων στο Microsoft SQL Server Management Studio.



Στη συνέχεια, διαμορφώσαμε τους πίνακες μας, λαμβάνοντας υπόψη τα χαρακτηριστικά και τα δεδομένα που περιλαμβάνει ο κάθε πίνακας. Παράλληλα, καθορίσαμε τις συνδέσεις μεταξύ των πινάκων (Foreign Keys) μέσω κατάλληλων SQL queries.



Τέλος, με SQL queries εισαγάγαμε τα δεδομένα από τα αρχεία CSV στους πίνακες της βάσης μας.

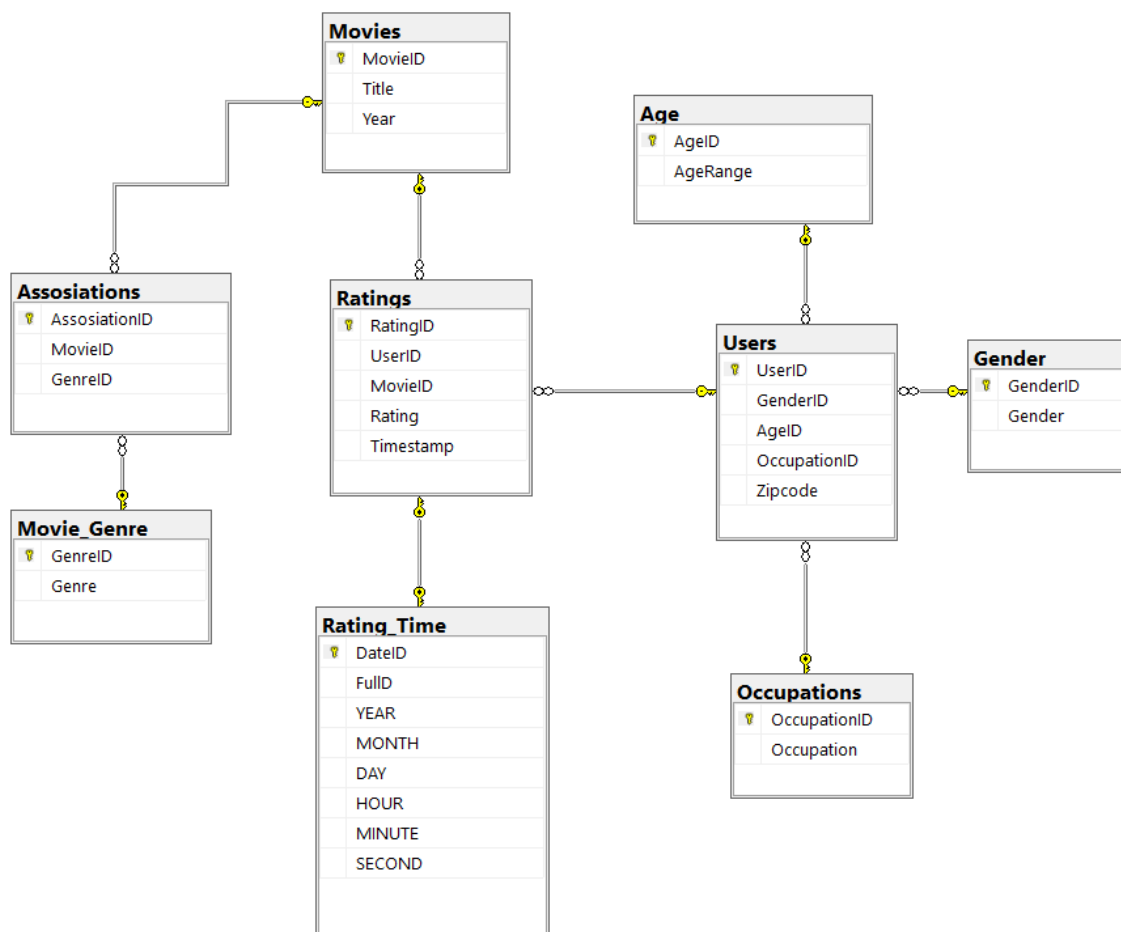


Συγκεκριμένα, για τον πίνακα των ταινιών, χρησιμοποιήσαμε τον ορισμό FIELDTERMINATOR='#', προκειμένου να αποφύγουμε πιθανά λάθη λόγω της ύπαρξης του χαρακτήρα «,» στον τίτλο ορισμένων ταινιών. Με αυτόν τον τρόπο διασφαλίσαμε τη σωστή εισαγωγή των δεδομένων χωρίς να επηρεαστεί η διαδικασία από ειδικούς χαρακτήρες στους τίτλους.

```
USE [MovieRatingDB]

BULK
INSERT [dbo].[Movies]
FROM 'C:\Users\30698\OneDrive\Desktop\ml-1m\movies.csv'
WITH
(
    FIELDTERMINATOR = '#',
    ROWTERMINATOR = '\n'
)
```

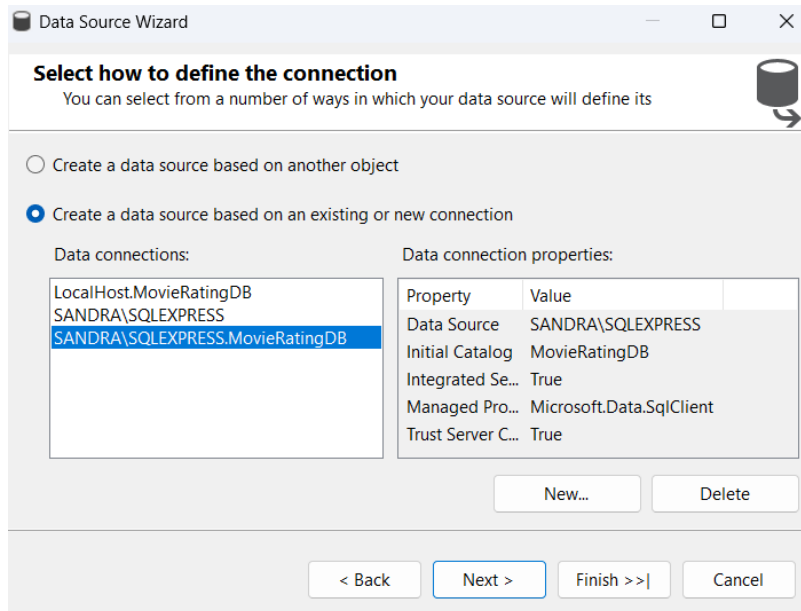
Έτσι, μετά από την εισαγωγή των δεδομένων στην βάση το Schema που δημιουργείται στον SQL Server είναι το εξής:



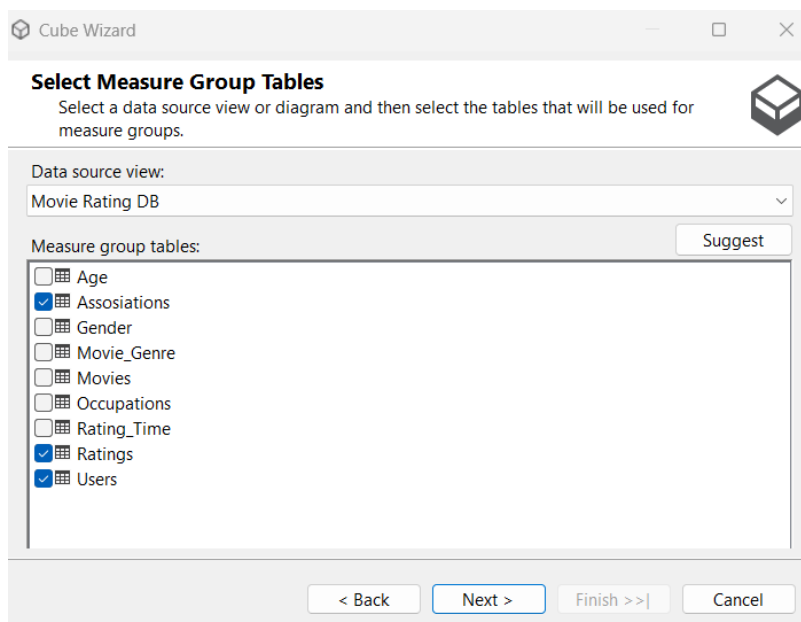
## Δημιουργία Cube

Για να δημιουργήσουμε τον κύβο των δεδομένων μας στο Visual Studio θα δημιουργήσουμε ένα Analysis Services Multidimensional Project.

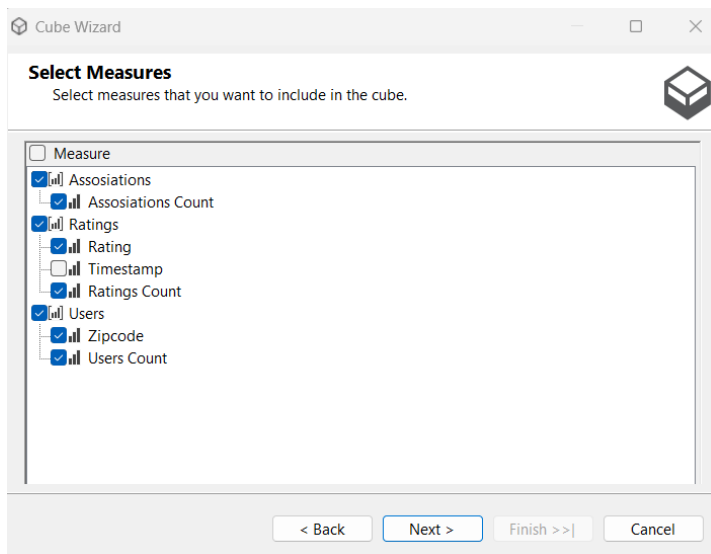
Στη συνέχεια, πατάμε στο data source και ανοίγει το παρακάτω Data Source Wizard και, έτσι, ξεκινάμε την υλοποίηση του κύβου.



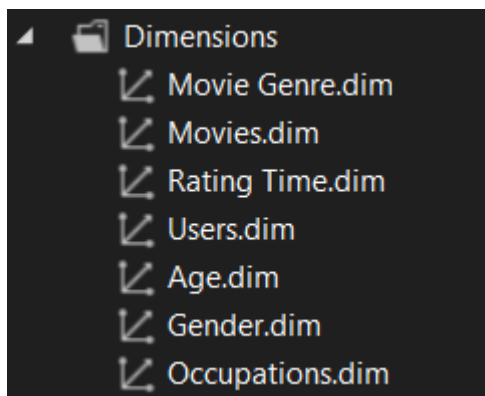
Για την δημιουργία του κύβου, επιλέξαμε για Measure Group Tables, τον πίνακα Ratings αφού αποτελεί τον Fact Table μας, τον Users αφού περιέχει σημαντικές πληροφορίες για την ανάλυση μας και τον Associations αφού αποτελεί ενδιάμεσος πίνακας.



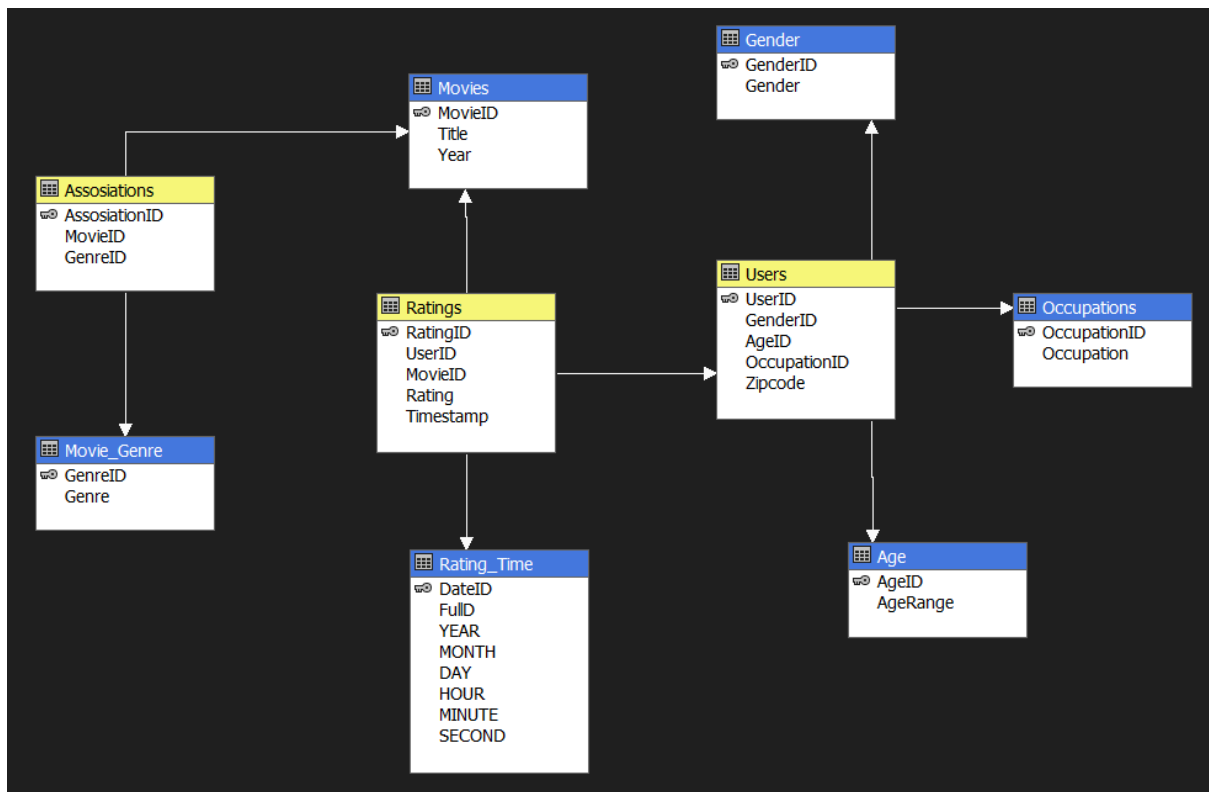
Στη συνέχεια επιλέξαμε τα measures που θέλουμε να μετράει ο κύβος μας, δηλαδή τα εξής:



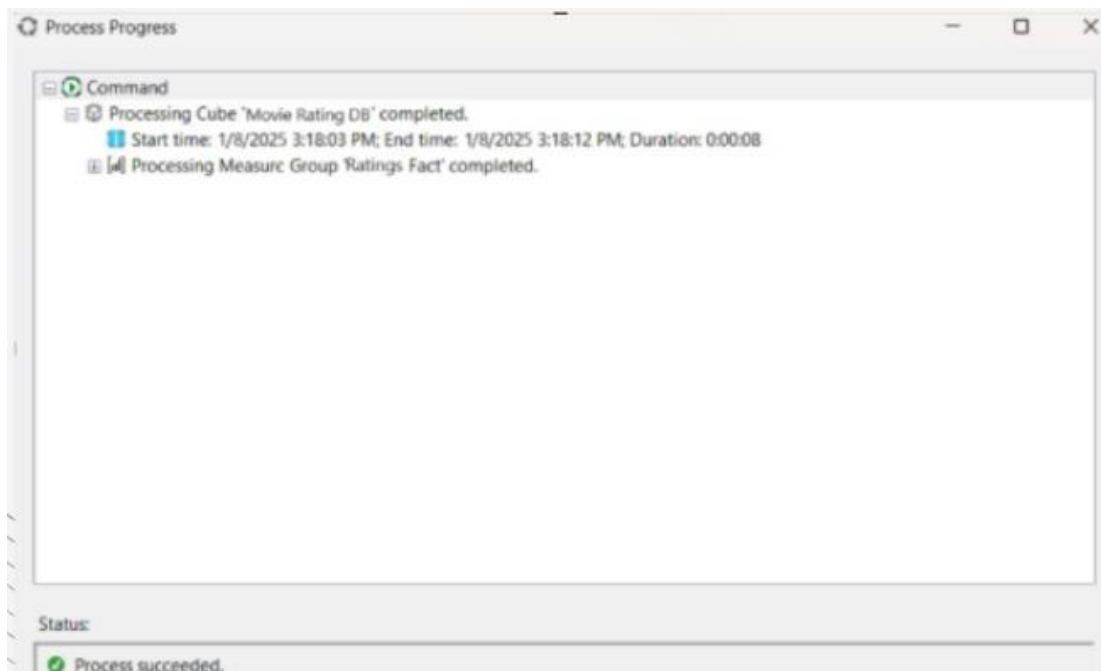
Οι διαστάσεις που προκύπτουν είναι οι παρακάτω. Η διάσταση **Movie Genre** περιγράφει το είδος κάθε ταινίας (π.χ., Κωμωδία, Δράμα, Δράση), ενώ η διάσταση **Movies** περιλαμβάνει πληροφορίες για κάθε ταινία, όπως τίτλος, έτος κυκλοφορίας και σύνδεση με το είδος. Η διάσταση **Rating Time** καταγράφει τη χρονική στιγμή της κάθε αξιολόγησης, περιλαμβάνοντας έτος, μήνα, ημέρα και ώρα. Η διάσταση **Users** περιλαμβάνει δημογραφικά στοιχεία χρηστών, όπως ηλικιακή ομάδα, επάγγελμα και τοποθεσία. Η διάσταση **Gender** αποθηκεύει το φύλο των χρηστών (π.χ., Άνδρας, Γυναίκα), ενώ η διάσταση **Occupations** περιγράφει τα επαγγέλματα των χρηστών (π.χ., Φοιτητής, Μηχανικός, Συνταξιούχος).



Έτσι μετά από τη συγκεκριμένη επεξεργασία ο κύβος που δημιουργήθηκε είναι ο εξής:



Κάναμε **με επιτυχία** process κύβο μας, μετά από επιτυχές deployment.



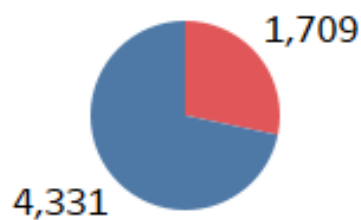


## Οπτικοποίηση με Tableau

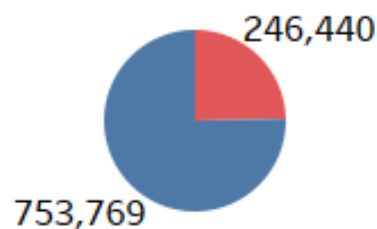
Μετά τη δημιουργία του κύβου δεδομένων, χρησιμοποιήσαμε το Tableau για τη δημιουργία dashboard και οπτικών αναπαραστάσεων. Αφού εισάγαμε τα δεδομένα μας από τον κύβο στο Tableau, δημιουργήσαμε την παρακάτω οπτική απεικόνιση.

Από την ανάλυση των δεδομένων, παρατηρήσαμε ότι από τους συνολικά 6.040 χρήστες, μόνο το 25% είναι γυναίκες. Παράλληλα, το ποσοστό των κριτικών που έχουν πραγματοποιηθεί από γυναίκες είναι αντίστοιχο, κυμαινόμενο επίσης γύρω στο 25%.

### Users per gender

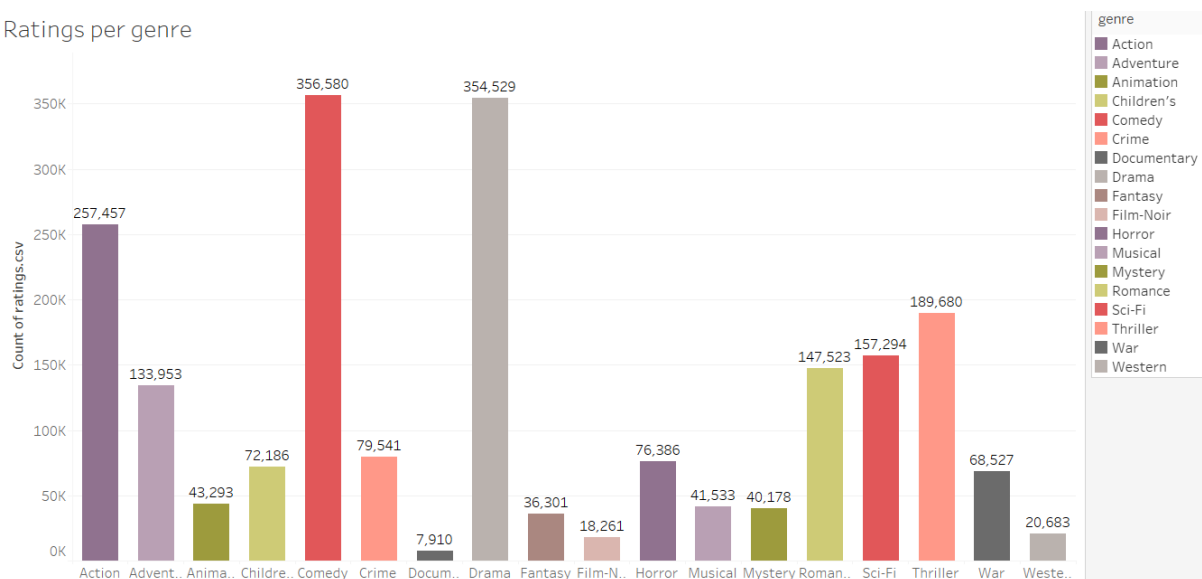


### Ratings per gender

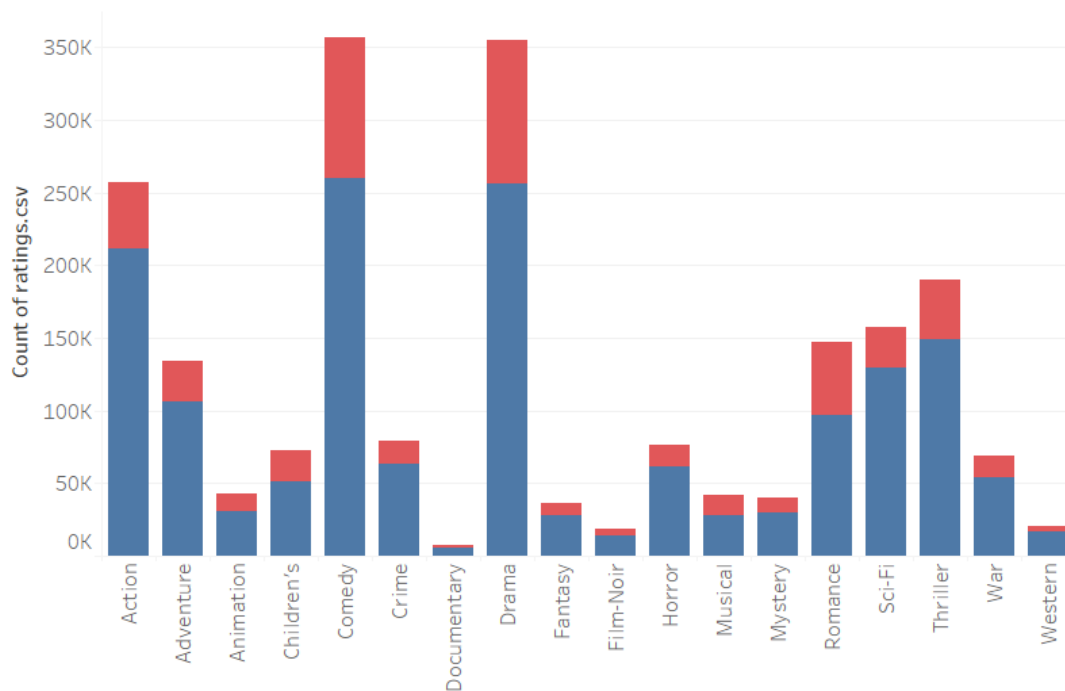


Επιπλέον, παρατηρήσαμε ότι οι κατηγορίες ταινιών με τις περισσότερες κριτικές είναι οι εξής: **Comedy**, **Drama** και **Action**.

### Ratings per genre

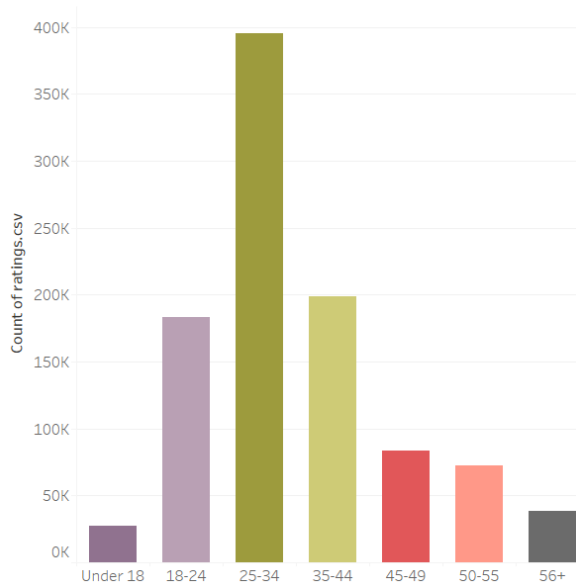


Ratings per movie genre and gender

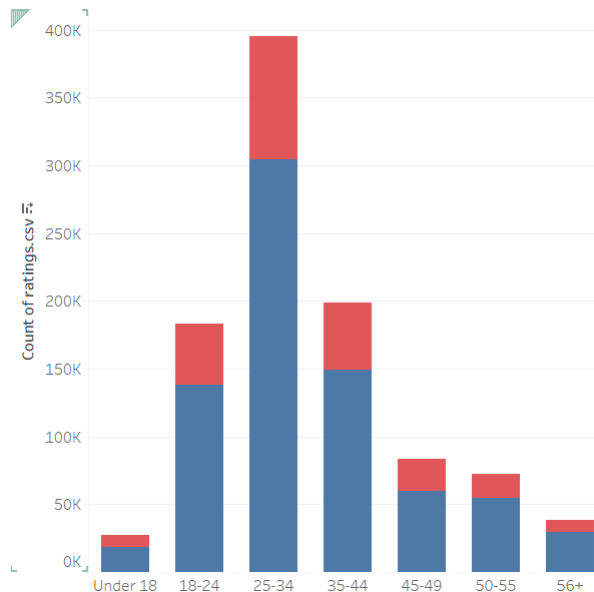


Ακόμα, οι ηλικιακές κατηγορίες που πραγματοποίησαν τις περισσότερες κριτικές είναι η κατηγορία 25-34 ετών, με περίπου 400.000 κριτικές, δηλαδή το 40% του συνόλου των κριτικών. Επιπλέον, κάναμε μια σύγκριση μεταξύ ανδρών και γυναικών στις κριτικές.

Ratings per age group and gender

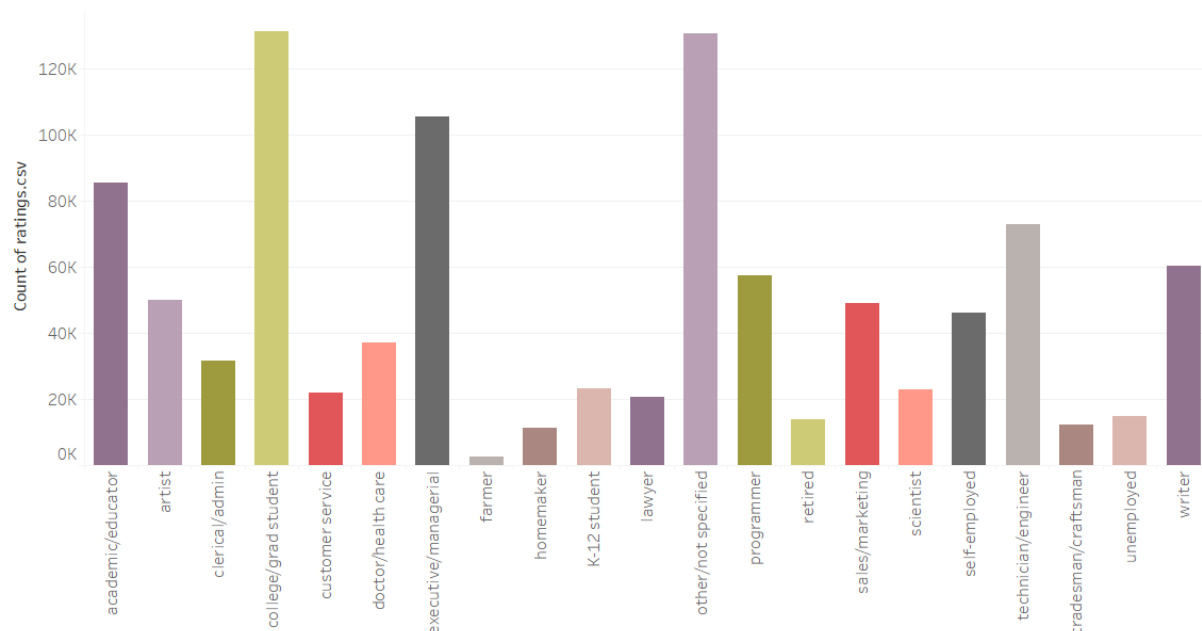


Ratings per age group and gender

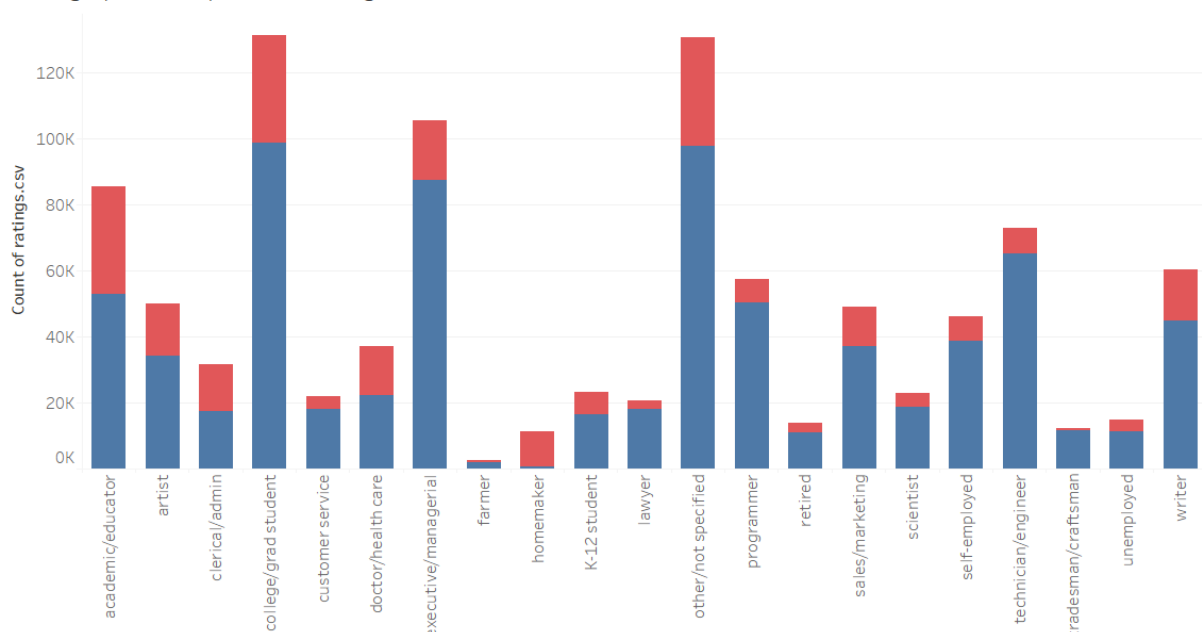


Οι κατηγορίες επαγγελματιών που πραγματοποίησαν τις περισσότερες κριτικές είναι οι εξής: **college/graduate student, executive/managerial** και **other/not specified**. Επιπλέον, κάναμε μια σύγκριση μεταξύ ανδρών και γυναικών στις κριτικές.

Ratings per occupation

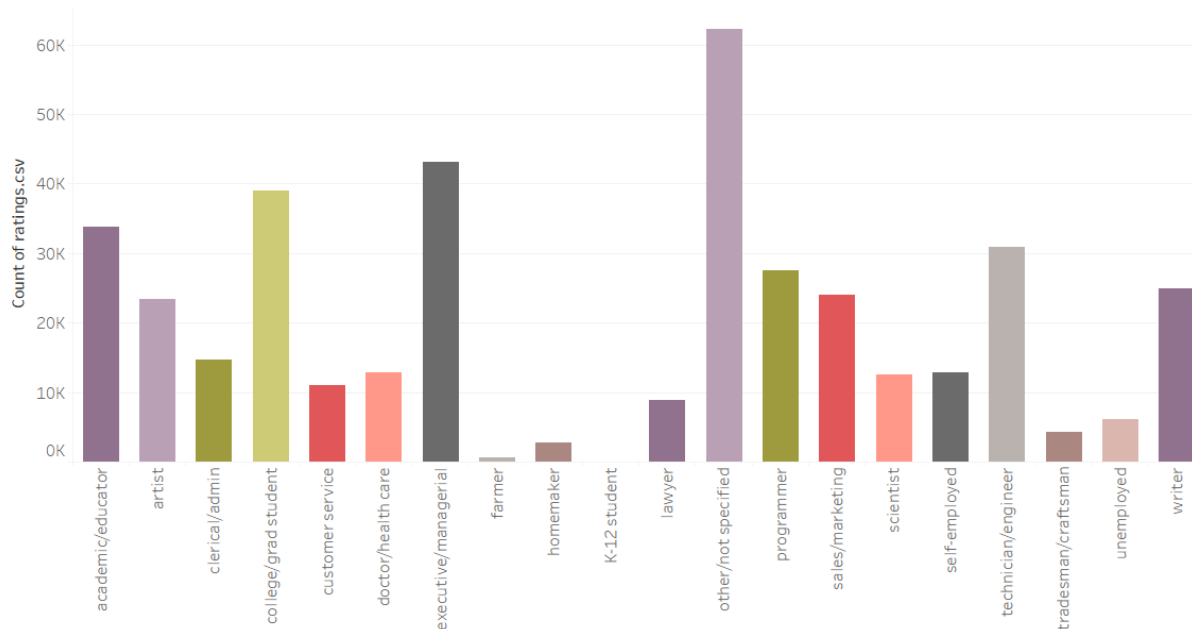


Ratings per occupation and age

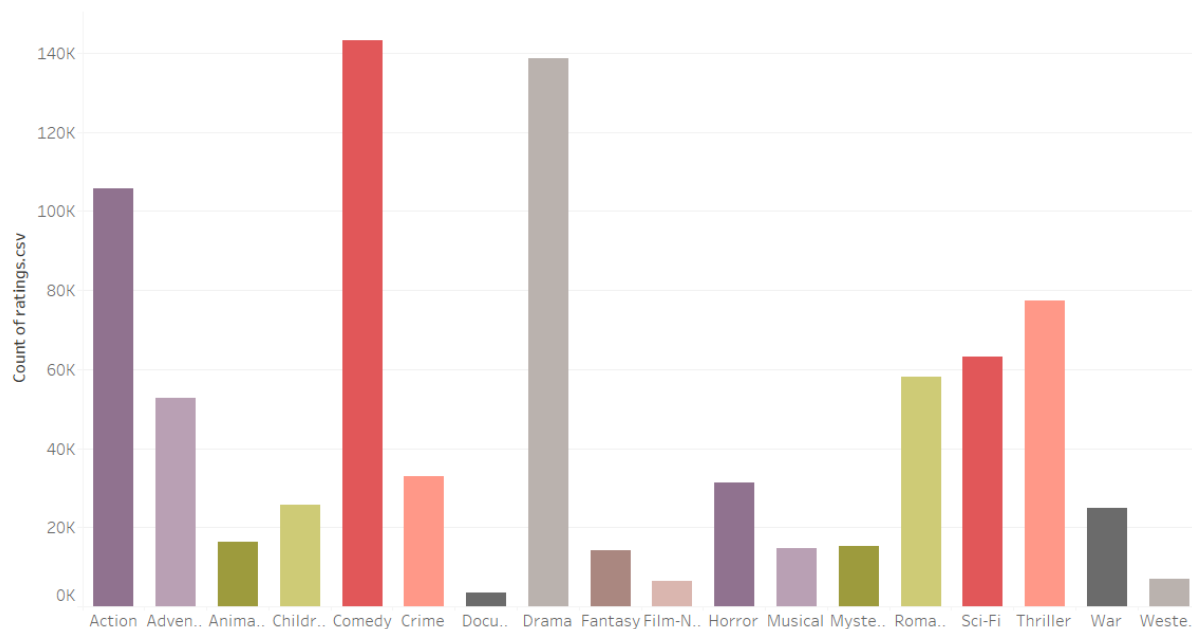


Συγκεκριμένα, για την ηλικιακή κατηγορία των 25-34 ετών, παρατηρούμε τους εξής αριθμούς κριτικών ανάλογα με τις επαγγελματικές κατηγορίες και το είδος της ταινίας:

Ratings per occupation in the age grou 25-34



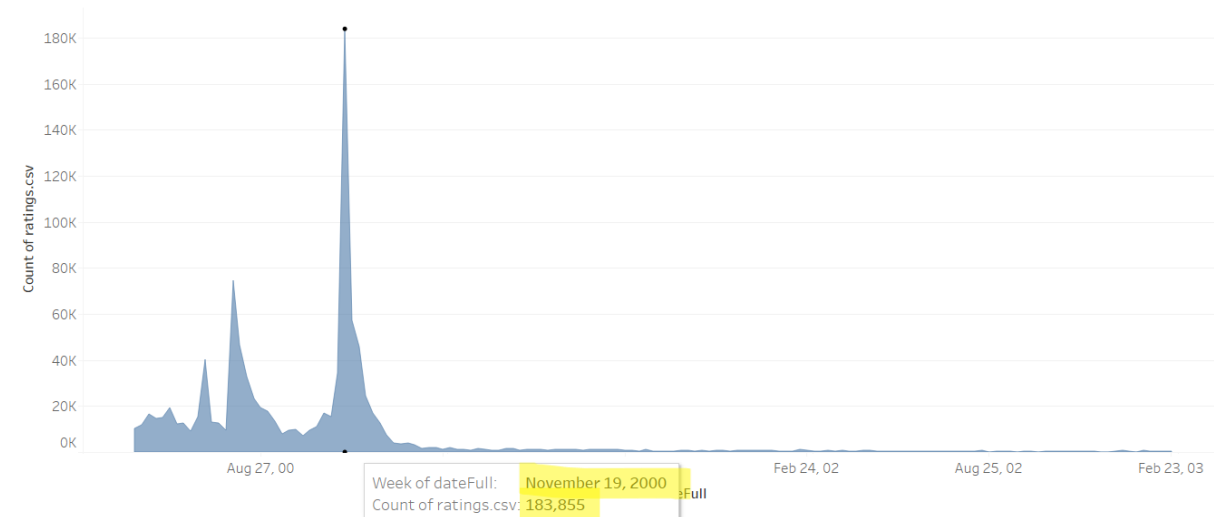
Ratings per genre in the age grou 25-34



Στο παρακάτω διάγραμμα παρατηρούμε τον αριθμό των κριτικών ανά εβδομάδα από το 2000 έως το 2003. Συγκεκριμένα, παρατηρούμε ότι το Νοέμβριο του 2000

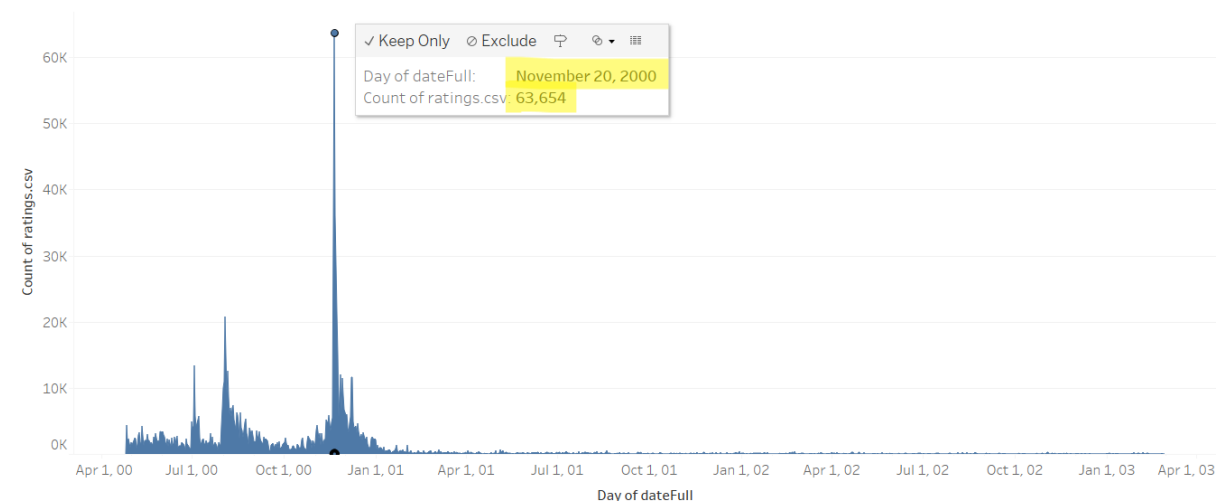
πραγματοποιήθηκαν οι περισσότερες κριτικές, με περίπου 183 χιλιάδες κριτικές. Επίσης στο διάγραμμα φαίνεται ότι το 2000 ήταν η χρονιά με τις περισσότερες κριτικές.

Ratings per week



Συγκεκριμένα, η ημερομηνία με τις περισσότερες κριτικές ήταν στις 20 Νοεμβρίου του 2000 με 63 χιλιάδες κριτικές.

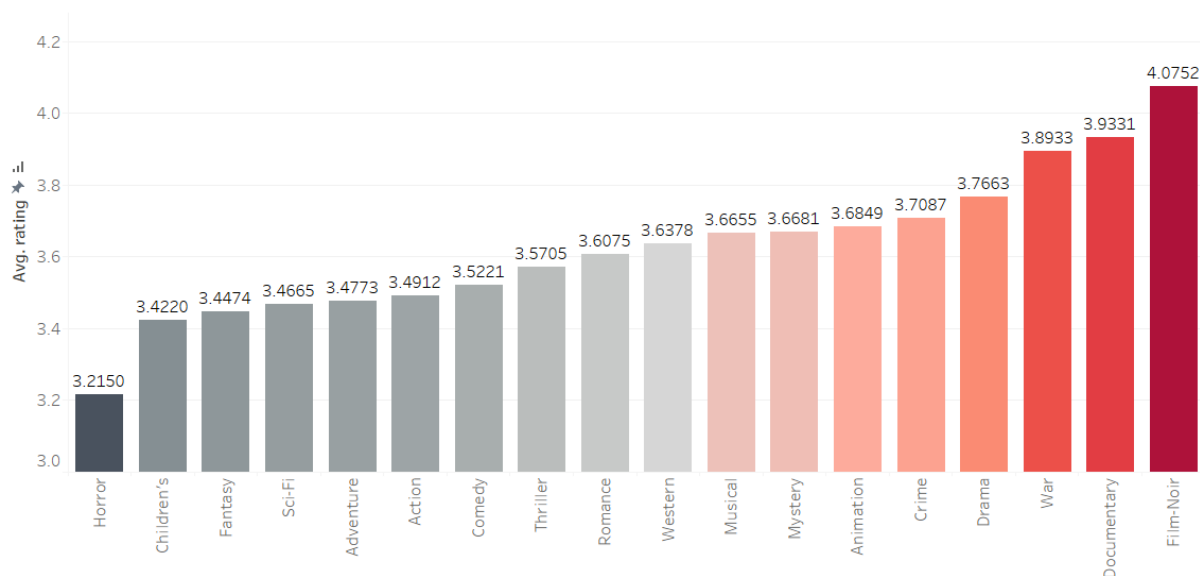
Ratings per day



Στη συνέχεια, δημιουργήσαμε οπτική απεικόνιση για να εξετάσουμε τη μέση αξιολόγηση ανά είδος ταινίας. Διαπιστώσαμε ότι όλες οι κατηγορίες είχαν συνολική μέση βαθμολογία πάνω από 3 αστέρια, δηλαδή πάνω από τη βάση. Συγκεκριμένα, η κατηγορία με την

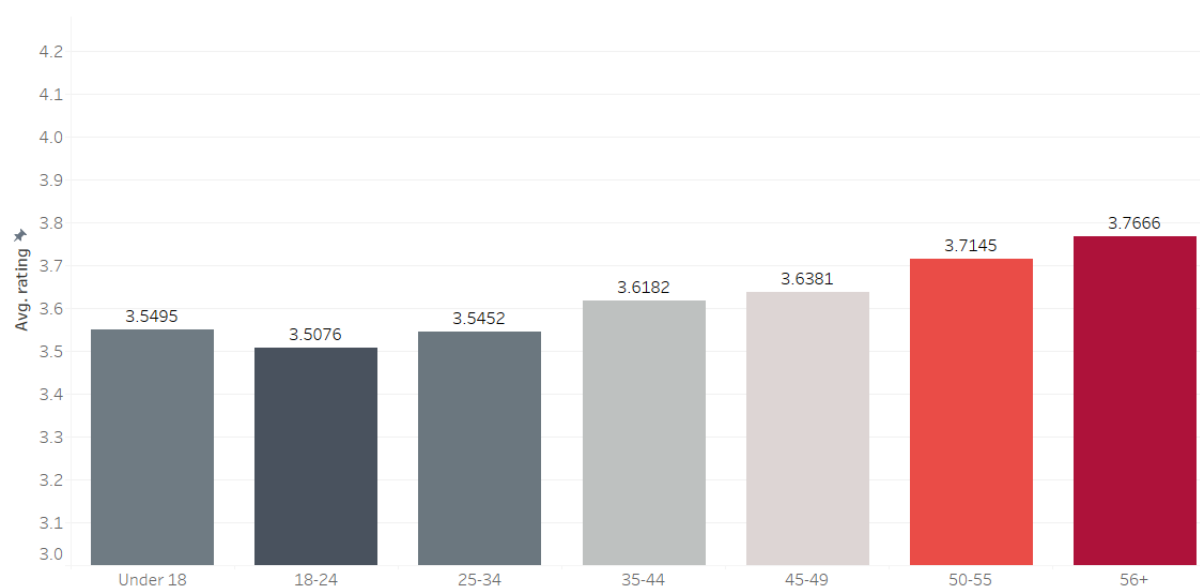
υψηλότερη μέση βαθμολογία, σύμφωνα με τις αξιολογήσεις των χρηστών, ήταν η **Film-Noir**, ενώ η κατηγορία με τη χαμηλότερη βαθμολογία ήταν η **Horror**.

Average Rating per Genre



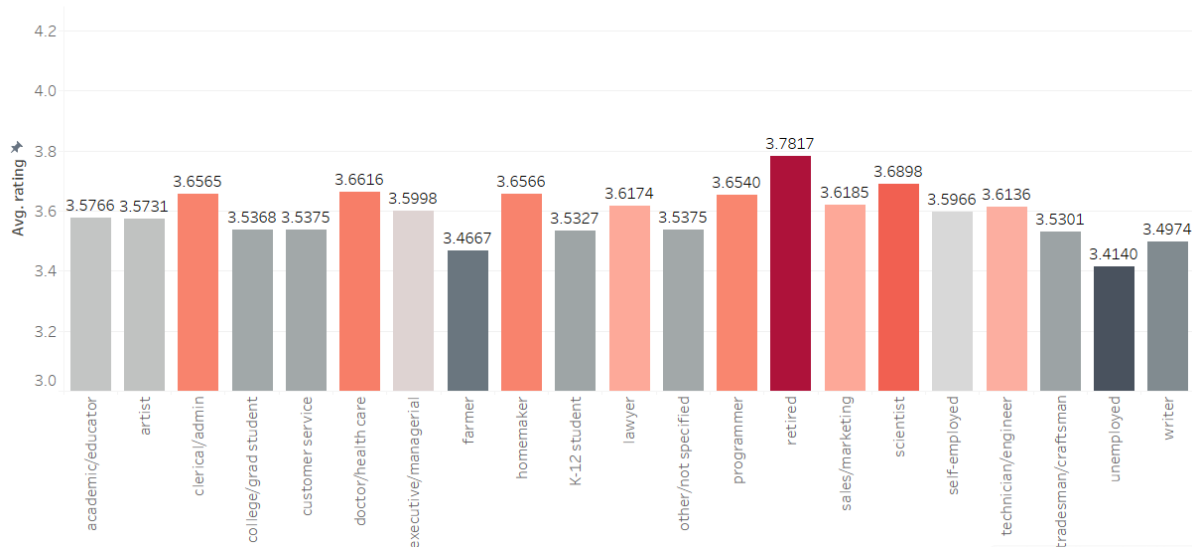
Ακόμα, παρατηρήθηκε ότι οι πιο **αυστηροί κριτές** ήταν τα άτομα ηλικίας **18-24**, ενώ τα άτομα **άνω των 56** ετών συνήθιζαν, αναλογικά, να δίνουν τις **υψηλότερες βαθμολογίες**.

Average Rating per Age Group



Ακόμα, ιδιαίτερο ενδιαφέρον παρουσιάζει το γεγονός ότι, όσον αφορά τις επαγγελματικές κατηγορίες, οι **συνταξιούχοι (retired)** έδωσαν κατά μέσο όρο τις καλύτερες βαθμολογίες, ενώ τις χειρότερες βαθμολογίες, αναλογικά, έδωσαν οι **άνεργοι (unemployed)**.

Average Rating per Occupation



Τέλος, όσον αφορά τη σύγκριση μεταξύ των φύλων, παρατηρούμε ότι οι **γυναίκες** συνήθιζαν να αφήνουν ελαφρώς καλύτερες κριτικές σε σχέση με τους **άνδρες**.

Average Rating per Gender

